

Цель: Проектирование и разработка мобильного приложения
«Обучающий контент»

Задачи, выделенные для достижения цели:

- Провести анализ и описание предметной области.
- Спроектировать и создать базу данных по заданной теме.
- Разработать алгоритм функционирования информационной системы.

- Создать дизайн и интерфейс приложения
- Разработать и реализовать приложение.
- Подготовить руководство пользователя по работе с приложением.

Приложение включает следующий функционал:

- Формирование и структурирование предоставляемого контента
- Авторизация и регистрация пользователей
- Связь с сервером
- Демонстрация контента в читабельном формате
- Статистика (Рейтинг контента)
- Архивирование устаревшего контента

Содержание	
Введение.....	4
1 Анализ предметной области.....	6
1.1 Предметная область	6
1.1.1 Описание предметной области	6
1.1.2 Организационная структура	8
1.1.3 Структурная схема приложения	10
1.1.4 Основные бизнес-процессы	12
1.2 Содержание автоматизируемых задач.....	14
1.2.1 Функции информационной системы и автоматизируемые задачи приложения.....	14
1.2.2 Диаграмма последовательности	15
2 Техническое проектирование.....	18
2.1 Проектирование структуры базы данных.....	18
2.2 Выбор и обоснование выбора программного средства разработки.	20
2.3 Проектирование пользовательского интерфейса	22
3 Рабочее проектирование	24
3.1 Разработка базы данных	24
3.2 Разработка пользовательского интерфейса	28
3.2 Тестирование программного продукта	32
4 Инструкции по работе с АИС	36
4.1 Требования к техническому обеспечению.....	36
4.2 Инструкция системного администратора, программиста.....	37

					ДП 09.02.07.21и1.16.000 ПЗ						
Изм.	Лист	№ докум.	Подпись	Дата							
Разраб.		Случаев М.К.			ПОЯСНИТЕЛЬНАЯ ЗАПИСКА ДП			Лит.	Лист	Листов	
Провер.		Мавлекеева Л. И.						v		2	62
								КАТК гр.			
Н. контр.											
Утв.											

4.3 Руководство пользователя	39
5 Охрана труда	46
Заключение.....	51
Список информационных источников	52
Приложение А.....	54
Приложение Б	57
Приложение В.....	59
Приложение Г	60
Приложение Д.....	62

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		3

Введение

В современную эпоху стремительного развития цифровых технологий образование активно трансформируется, переходя на новые форматы взаимодействия. Цифровые инструменты играют ключевую роль в учебном процессе, обеспечивая оперативный доступ к материалам, упрощая коммуникацию между студентами и преподавателями, а также способствуя формированию единого образовательного пространства.

Тем не менее, несмотря на наличие множества платформ и мессенджеров, обмен лекциями, конспектами и другими учебными материалами остаётся слабо структурированным. Часто информация передаётся в разрозненной форме — через чаты, электронную почту или облачные хранилища — что затрудняет систематизацию знаний, поиск нужных данных и подготовку к занятиям. Это приводит к потере важных материалов и снижению эффективности учебного процесса.

В связи с этим особенно актуальной становится задача разработки специализированного мобильного приложения, направленного на упрощение обмена образовательным контентом. Предлагаемое решение — мобильное приложение «Обучающий контент» — будет служить централизованной платформой для хранения, поиска и распространения учебных материалов, обеспечивая доступность и упорядоченность информации.

Цель проекта:

Проектирование и разработка мобильного приложения «Обучающий контент», направленного на оптимизацию образовательного процесса и улучшение доступности учебных ресурсов.

Основные задачи для достижения цели:

- Провести комплексный анализ предметной области и выявить ключевые потребности пользователей.
- Спроектировать и реализовать реляционную базу данных для хранения и структурирования контента.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		4

- Разработать алгоритмы и архитектуру функционирования информационной системы.
- Создать интуитивно понятный и адаптивный пользовательский интерфейс.
- Осуществить полную разработку и тестирование мобильного приложения.
- Подготовить подробное руководство пользователя по установке и использованию приложения.

Реализация данного проекта позволит значительно упростить организацию учебной деятельности, сократить временные затраты на поиск информации и повысить качество подготовки студентов за счёт лучшего доступа к необходимым учебным материалам.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		5

1 Анализ предметной области

1.1 Предметная область

1.1.1 Описание предметной области

Образовательные учреждения играют важную роль в подготовке специалистов, обеспечивая их необходимыми знаниями и навыками для будущей профессиональной деятельности. Современные технологии активно интегрируются в учебный процесс, способствуя его совершенствованию и повышению качества образования. Однако многие учебные заведения сталкиваются с трудностями, связанными с организацией учебного процесса, доступностью образовательных материалов и эффективным взаимодействием между студентами и преподавателями.

Ключевые направления деятельности образовательных организаций включают:

- проведение лекций, семинаров и практических занятий;
- разработку и реализацию образовательных программ;
- оценку знаний студентов и проведение экзаменов;
- организацию научных исследований и проектной деятельности;
- взаимодействие с работодателями для содействия трудоустройству выпускников;
- обеспечение доступа к учебным материалам и методическим пособиям.

Основная цель образовательных учреждений — обеспечение студентов качественным образованием, способствующим их профессиональному и личностному развитию. Достижение этой цели требует не только квалифицированного преподавательского состава, но и активного использования современных информационных технологий, упрощающих процесс обучения и расширяющих доступ к образовательным ресурсам.

Функции образовательных организаций включают:

- обучение студентов в соответствии с утвержденными программами;

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		6

- предоставление доступа к учебным материалам;
- контроль успеваемости и проведение аттестации;
- организацию мероприятий для повышения квалификации и обмена опытом;

- создание условий для научных исследований и проектной работы.

В рамках учебного процесса студент выполняет следующие действия:

- посещает лекционные и практические занятия;
- изучает учебные материалы и выполняет задания;
- проходит промежуточную и итоговую аттестацию;
- участвует в научных и исследовательских проектах;
- взаимодействует с преподавателями и сокурсниками;
- готовится к будущей профессиональной деятельности, в том числе через стажировки и практики.

Использование современных технологий в образовательной среде способствует улучшению доступа к учебным материалам, упрощает взаимодействие между участниками учебного процесса и повышает эффективность обучения.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлекеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		7

1.1.2 Организационная структура

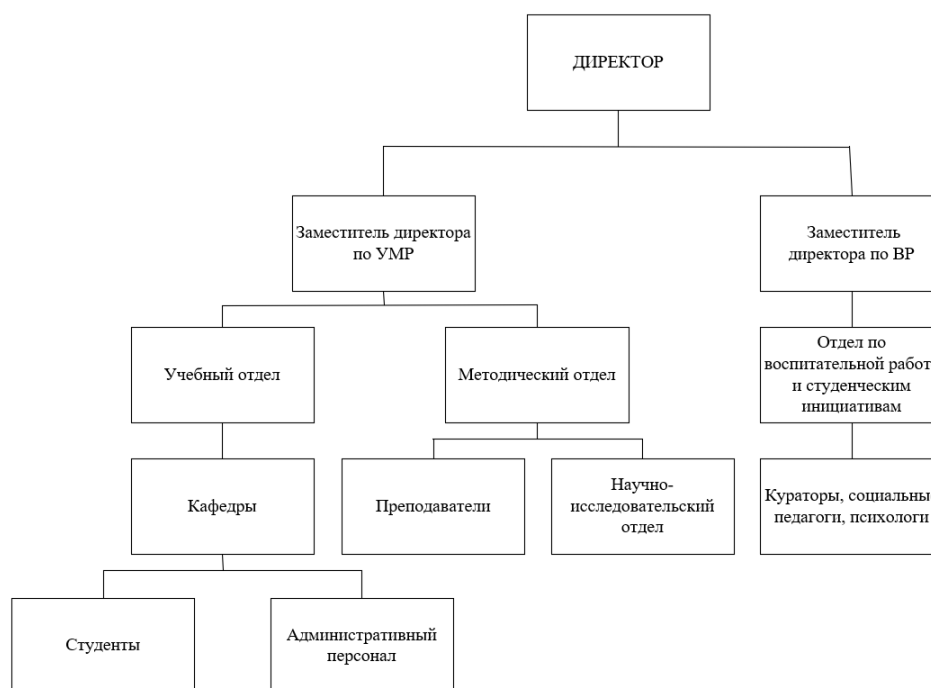


Рисунок 1- Организационная структура

Заместитель руководителя по учебной работе отвечает за координацию и проведение образовательного процесса. В его компетенцию входит разработка учебных планов, контроль над качеством преподавания, внедрение передовых технологий в обучении и организация экзаменационных процедур студентов. Под его началом работают отделы, отвечающие за учебный процесс и методику, а также кафедры. Учебный отдел занимается составлением расписания, организацией экзаменационных сессий, ведением учета успеваемости студентов и оформлением официальных документов, включая дипломы и зачётные книжки. Методический отдел разрабатывает учебные материалы, внедряет инновационные образовательные подходы и следит за соблюдением образовательных стандартов. Кафедры объединяют преподавателей по специальностям, обеспечивают реализацию учебных программ, курируют дипломные и курсовые работы студентов, а также участвуют в научно-исследовательской деятельности.

Преподаватели играют ключевую роль в образовательном процессе, проводя лекции, семинары, практические и лабораторные работы, оценивая знания студентов и консультируя их по учебным вопросам. Кроме того,

научно-исследовательский отдел организует научные исследования, конференции, занимается привлечением грантов и сотрудничает с внешними научными организациями.

Заместитель директора по воспитательной работе отвечает за вопросы, связанные с дисциплиной, психологическим климатом, внеклассной деятельностью студентов и их социальной адаптацией. В его ведении находится отдел по воспитательной работе, который занимается организацией культурно-массовых и спортивных мероприятий, поддержкой студенческих инициатив и взаимодействием с кураторами, социальными педагогами и психологами. Эти специалисты помогают студентам в адаптации, решении личных и учебных проблем, проводят тренинги и консультации.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		9

1.1.3 Структурная схема приложения

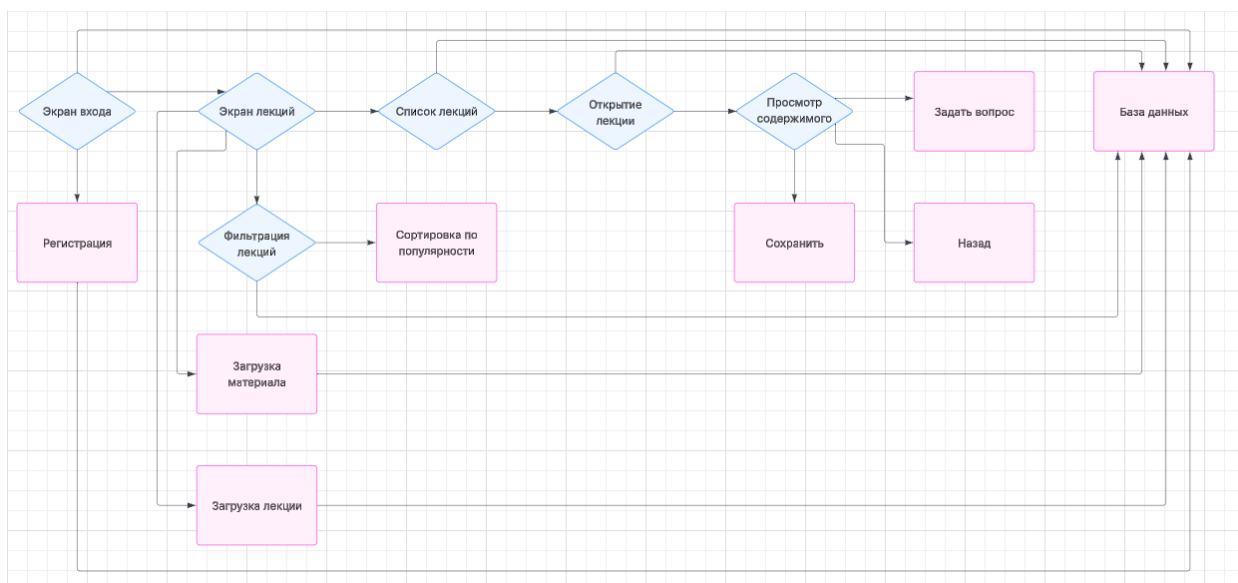


Рисунок 2 - структурная схема приложения

Основные этапы (ромбы — точки перехода):

1) Экран входа

- Позволяет пользователю авторизоваться или перейти к регистрации.

2) Экран лекций

- Центральный узел, откуда пользователь может:
 - Открыть список лекций
 - Фильтровать лекции
 - Сортировать по популярности
 - Загрузить материал или добавить лекцию

3) Просмотр содержимого

- Показывает текст лекции, даёт возможность:
 - Сохранить лекцию на устройство
 - Задать вопрос преподавателю
 - Вернуться назад

Функциональные действия (прямоугольники):

- Регистрация — доступна с экрана входа.
- Фильтрация лекций — помогает сузить выбор.

– Сортировка по популярности — дополнительный способ упорядочивания.

– Добавление лекции — для добавления нового контента.

– Загрузка материала — для чтения контента.

Поток пользователя:

– Пользователь входит или регистрируется.

– Переходит к экрану лекций.

– Ищет нужную лекцию (фильтрация/сортировка).

– Загружает или читает лекции.

– При чтении может сохранить, задать вопрос или вернуться.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлекеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		11

1.1.4 Основные бизнес-процессы

Формирование и загрузка лекционного контента

- Загрузка файлов (PDF)
- Назначение метаданных (название, курс, предмет)

- Классификация по курсам и предметам

Структурирование контента в базе данных

- Хранение и организация контента по категориям
- Поддержка поиска и фильтрации

Синхронизация и связь с сервером

- Обработка запросов от клиента
- Передача данных через API
- Обеспечение безопасности и отказоустойчивости

Демонстрация контента пользователю

- Отображение материалов в адаптированном формате
- Поддержка навигации, масштабирования
- Интерфейс для чтения и взаимодействия с лекциями

Архивирование устаревшего контента

- Определение неактуальных материалов
- Перемещение в архив

Управление пользователями и доступом к материалам

- Назначение ролей (студент, преподаватель, администратор)

Поиск и фильтрация лекций

- Реализация текстового поиска
- Фильтрация курсу, рейтингу и предмету
- Быстрый доступ к нужной информации

Диаграмма прецедентов



Рисунок 3 - Диаграмма прецедентов

Пользователь включает в себя преподавателя и студента

Экторы:

- Студент — пользователь, изучающий материалы, а также загружающий их.
- Преподаватель — пользователь, который может добавлять материалы.
- Администратор — пользователь с расширенными правами, включающий функции всех акторов и возможность архивации контента, а также проверяет и одобряет материал, загруженный студентами.

Общие прецеденты для Студента и Преподаватель:

- Авторизация / Регистрация — акторы входят в систему или регистрируются для получения доступа.
- Просмотр контента — позволяет пользователю открывать и читать лекции в приложении.

– Поиск и фильтрация материалов — предоставляет возможность находить нужные лекции, по ключевым словам, темам, рейтингу и другим критериям.

– Загрузка лекционного материала — позволяет загружать обучающий контент в систему, указывая необходимые метаданные (название, курс, предмет, сам файл с лекцией), если файл загружается студентом, то перед тем, как файл будет предоставлен в общий доступ он должен быть одобрен администратором.

– Загрузка лекционного материала на устройство — позволяет загрузить просматриваемый материал в загрузки устройства для offline доступа.

Дополнительный прецедент для Студента:

– Уточнение непонятного фрагмента у преподавателя — студент при встрече с непонятным участком лекции имеет возможность отправить вопрос преподавателю на почту.

Дополнительные прецеденты только для Администратора:

– Архивирование контента — Администратор может перемещать устаревшие материалы в архив, чтобы они не отображались пользователям в основном каталоге, но при этом сохранялись в системе.

– Одобрение контента — материал, загружаемый студентом, одобряется администратором в личном кабинете.

1.2 Содержание автоматизируемых задач

1.2.1 Функции информационной системы и автоматизируемые задачи приложения.

Функции информационной системы:

Управление пользователями

– Регистрация и авторизация студентов, учителей и администраторов

– Разграничение прав доступа в зависимости от роли

Обработка и хранение контента

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		14

- Загрузка лекционного материала
- Сохранение файлов и метаданных в базе данных
- Структурирование контента по темам, дисциплинам

Предоставление доступа к контенту

- Отображение, загрузка материалов в удобном и читаемом формате
- Поддержка фильтрации и поиска по ключевым параметрам

Управление актуальностью информации

- Выделение и перемещение устаревшего контента в архив

Формирование статистики и аналитики

- Подсчет просмотров

Обеспечение связи между клиентом и сервером

- Реализация REST API для обмена данными
- Синхронизация действий пользователей с серверной частью

1.2.2 Диаграмма последовательности

Диаграмма последовательности

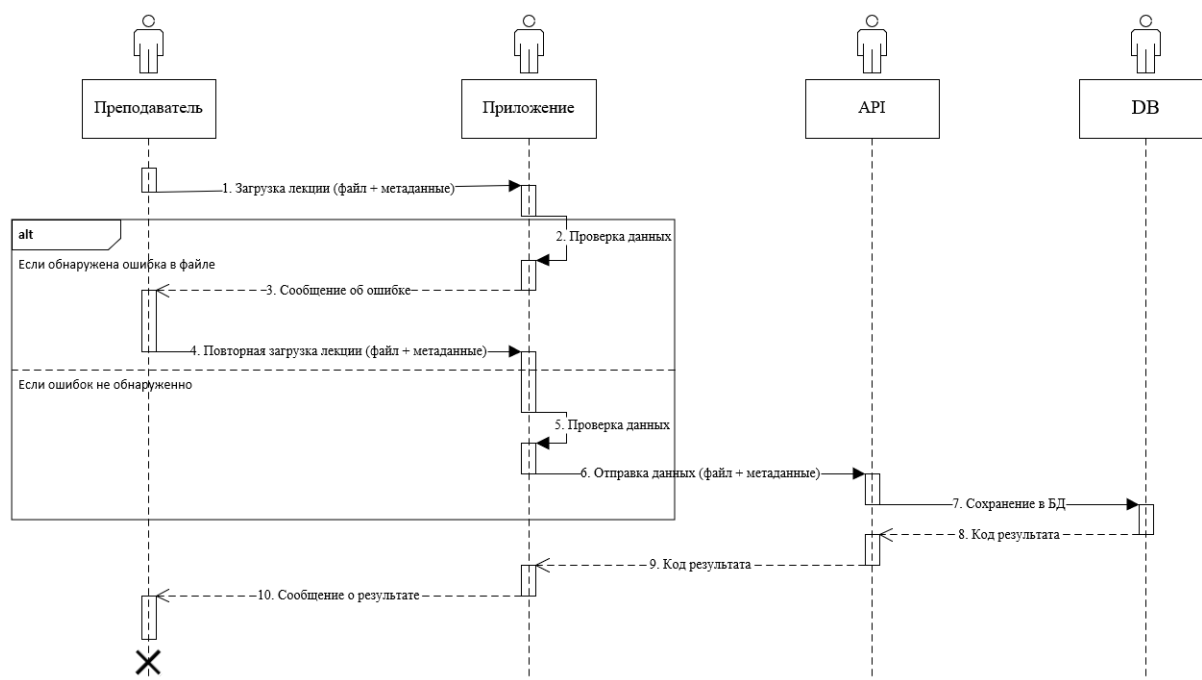


Рисунок 4 - Диаграмма последовательности для функции загрузки лекции преподавателем

Описание этапов:

- 1) Преподаватель инициирует загрузку лекции

- Преподаватель загружает файл лекции (обязательно с расширением PDF).

- Указывает метаданные:

- Название лекции (например, "Введение в алгоритмы"),

- Предмет (выбирает из выпадающего списка),

- Курс (указывается только цифра курса)

- Нажимает кнопку "Загрузить".

2) Приложение проверяет данные

- Проверка №1: Формат файла

- Приложение проверяет, что файл имеет расширение .pdf.

- Проверка №2: Заполненность метаданных

- Приложение проверяет, что все поля (Название, Предмет, Курс) заполнены.

Если проверки не пройдены (файл не PDF):

- Приложение возвращает ошибку:

```
{  
  "status": "error",  
  "message": "Ошибка: файл не PDF / не заполнены обязательные  
поля"  
}
```

- Преподаватель получает уведомление и должен повторить загрузку.

3) Повторная загрузка (если были ошибки)

- Преподаватель исправляет ошибки (загружает PDF-файл).

- Повторяет шаг 1.

4) Приложение отправляет данные в API

- Если проверки пройдены, приложение формирует запрос к API:

```
{  
  "file": "лекция_1.pdf",  
  "metadata": {
```

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		16


```

    "name_lecture": "Введение в алгоритмы",
    "lecture": "Программирование",
    "course": "3"
  }
}

```

– Запрос отправляется через HTTP POST на эндпоинт API (/api/find_lecture/)

5) API сохраняет данные в БД

- API получает запрос, извлекает файл и метаданные.
- Генерирует уникальный ID для лекции (Используется счётчик).
- Сохраняет в БД

6) Подтверждение от БД

- БД возвращает API подтверждение об успешном сохранении:
{"status": "400"}

7) Уведомление преподавателя

- API передает ответ приложению:
{"status": "400"}

– Приложение отображает преподавателю сообщение: "Лекция успешно загружена".

2 Техническое проектирование

2.1 Проектирование структуры базы данных

Словарь данных

Таблица user (Пользователи)

Таблица 1 - Таблица user

Поле	Тип	Описание
id_user	int	Уникальный идентификатор пользователя РК
surname	varchar	Фамилия пользователя
name	varchar	Имя пользователя
mail	varchar	Почта пользователя
type_user_id	int	Идентификатор типа пользователя (внешний ключ на user_type)
specialization_id	int	Идентификатор специальности (внешний ключ на specialization)
course	int	Курс, на котором учится пользователь
password	varchar	Пароль пользователя

Таблица user_type (Типы пользователей)

Таблица 2 - Таблица user_type

Поле	Тип	Описание
id_type_user	int	Уникальный идентификатор типа пользователя РК
type_user_name	varchar	Название типа пользователя (администратор, студент, преподаватель)

Таблица specialization (Специальности)

Таблица 3 - Таблица specialization

Поле	Тип	Описание
id_specialization	int	Уникальный идентификатор специальности РК
specialization_name	varchar	Название специальности

Таблица lecture (Лекции)

Таблица 4 - Таблица lecture

Поле	Тип	Описание
id_lecture	int	Уникальный идентификатор лекции РК
name	varchar	Название лекции
object_id	int	Идентификатор предмета (внешний ключ на object)
file_pdf	bytea	PDF-файл лекции
course	int	Курс, к которому относится лекция
user_id	int	Идентификатор пользователя (автора лекции, внешний ключ на user)

Поле	Тип	Описание
count_view	int	Количество просмотров лекции
status	int	Статус лекции

Таблица object (Предметы)

Таблица 5 - Таблица object

Поле	Тип	Описание
id_object	int	Уникальный идентификатор предмета РК
object_name	varchar	Название предмета

2.2 Выбор и обоснование выбора программного средства разработки

Для разработки программного обеспечения было выбрано несколько технологий, каждая из которых соответствует своей роли в архитектуре приложения. Выбор сделан на основе удобства разработки, надёжности и совместимости компонентов между собой.

Для создания мобильного приложения был выбран язык программирования Java. Это один из официальных языков для разработки под операционную систему Android, активно поддерживаемый Google. Java обладает широкой экосистемой библиотек, большим сообществом и высоким уровнем совместимости с Android Studio — основной средой разработки мобильных приложений. Кроме того, Java имеет хорошую производительность, устойчив к ошибкам за счёт строгой типизации и поддерживает обратную совместимость, что позволяет использовать его на разных версиях Android. Всё это делает Java оптимальным выбором для создания мобильного клиента приложения.

В качестве языка программирования для серверной части, реализующей API, выбран Python с использованием микрофреймворка Flask. Flask — это

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		20

лёгкий, но гибкий фреймворк, который отлично подходит для построения RESTful API. Python отличается лаконичным синтаксисом, что позволяет ускорить процесс разработки и упростить поддержку кода. Также Python имеет широкую поддержку библиотек и модулей, включая средства работы с базами данных, сериализацией, авторизацией и другими компонентами веб-приложений. Использование Flask делает API легко расширяемым и масштабируемым, а взаимодействие с базой данных через ORM существенно упрощает доступ к данным.

Для хранения и управления данными выбрана реляционная система управления базами данных PostgreSQL. Это мощная и надёжная СУБД с открытым исходным кодом, которая обеспечивает высокий уровень безопасности, производительности и соответствие стандартам ACID. PostgreSQL предоставляет широкие возможности по работе с данными, включая поддержку JSON, транзакций, триггеров, полнотекстового поиска и других продвинутых функций. Она отлично масштабируется и легко интегрируется с Python-приложениями, что делает её подходящим выбором для серверной части.

Для визуального управления базой данных используется pgAdmin — официальный инструмент для работы с PostgreSQL. Это удобное средство с графическим интерфейсом, позволяющее управлять таблицами, выполнять SQL-запросы, отслеживать состояния транзакций, проводить резервное копирование и восстановление данных. pgAdmin доступен как в виде десктопного, так и веб-приложения, и предоставляет все необходимые инструменты для эффективной работы с базой данных.

Таким образом, выбор Java, Python Flask, PostgreSQL и pgAdmin позволяет создать надёжную, расширяемую и поддерживаемую архитектуру приложения, при этом ускоряя разработку и снижая вероятность ошибок.

2.3 Проектирование пользовательского интерфейса



Рисунок 5 - Пользовательский интерфейс

На представленной схеме отображена структура пользовательского интерфейса мобильного приложения "Обучающий контент", которая определяет взаимодействие пользователей с различными функциональными модулями системы. Интерфейс строится в зависимости от роли пользователя — студент, преподаватель или администратор.

После запуска приложения все пользователи проходят этап авторизации или регистрации. В зависимости от введенных данных, система определяет роль пользователя и перенаправляет его на соответствующий главный интерфейс. Студент и преподаватель получают доступ к одинаковому пользовательскому представлению, тогда как администратор — к расширенному интерфейсу с дополнительными возможностями.

Из главного интерфейса пользователи могут перейти к просмотру списка доступных лекций. Это окно представляет собой каталог всех загруженных материалов, где можно воспользоваться фильтрацией, сортировкой или поиском. Выбрав нужный элемент из списка, пользователь открывает окно

просмотра лекции — отдельный экран, предназначенный для изучения содержания лекционного материала в читаемом формате, а также загрузки его на устройство и связи с преподавателем.

В главном интерфейсе доступна форма загрузки новой лекции. Это окно позволяет прикрепить файл, ввести название, курс, предмет лекции. После отправки данные обрабатываются и сохраняются в системе, становясь доступными другим пользователям, материал, загружаемый студентом, дополнительно проверяется преподавателем.

Дополнительно, администратор имеет возможность перехода к окну архивирования устаревших лекций, а также удаления и подтверждения лекций, загруженных студентами.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлекеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		23

3 Рабочее проектирование

3.1 Разработка базы данных

В рамках реализации проекта было принято решение использовать в качестве системы управления базами данных PostgreSQL. Этот выбор обусловлен высокой надёжностью, производительностью и гибкостью данной СУБД, а также её широкой распространённостью и активным сообществом поддержки. PostgreSQL отлично подходит для работы с большими объёмами структурированной информации и предоставляет широкие возможности для масштабирования и расширения функциональности при необходимости.

Для развертывания базы данных был арендован удалённый сервер, работающий под управлением операционной системы Ubuntu Server. Это надёжная и удобная в обслуживании платформа, которая обеспечивает стабильную работу всех компонентов и совместимость с необходимым программным обеспечением. Установка PostgreSQL производилась стандартным способом через пакетный менеджер APT, при помощи команды `sudo apt install postgresql`. После завершения установки было произведено первичное конфигурирование системы и проверка корректной работы службы.

Одним из важных этапов настройки стало обеспечение возможности удалённого подключения к базе данных, поскольку к ней должно обращаться API мобильного приложения. С этой целью был отредактирован конфигурационный файл `postgresql.conf`, где параметру `listen_addresses` было присвоено значение `*`, что разрешает приём подключений со всех IP-адресов. Этот шаг необходим для обеспечения гибкого доступа к базе из различных точек.

Далее был настроен файл `pg_hba.conf`, отвечающий за правила аутентификации и авторизации клиентов. В него была добавлена строка `host all all 0.0.0.0/0 md5`, разрешающая подключение к базе данных всем пользователям по паролю. Хотя данная настройка даёт максимальную гибкость, в целях безопасности в дальнейшем планируется ограничить диапазон допустимых IP-адресов. Также, для обеспечения корректного

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		24

функционирования соединений, через системный брандмауэр был открыт порт 5432, по которому PostgreSQL по умолчанию принимает входящие подключения. Команда `sudo ufw allow 5432/tcp` позволила разрешить этот трафик на уровне сетевого фильтра.

После завершения всех вышеописанных настроек служба PostgreSQL была перезапущена для применения новых параметров.

Для начала работы с базой данных через графический интерфейс PgAdmin необходимо установить соединение с сервером PostgreSQL. Сначала запустить PgAdmin. После открытия интерфейса щёлкнуть правой кнопкой мыши по разделу "Servers" в левой панели и выбрать пункт "Create > Server...". В появившемся окне на вкладке "General" указать имя подключения, "diplom", после чего во вкладке "Connection" ввести параметры: IP-адрес и имя сервера, порт (по умолчанию 5432), имя базы данных (diplom), а также логин и пароль для подключения. После заполнения всех данных нажать кнопку "Save", и дождаться подключения к серверу.

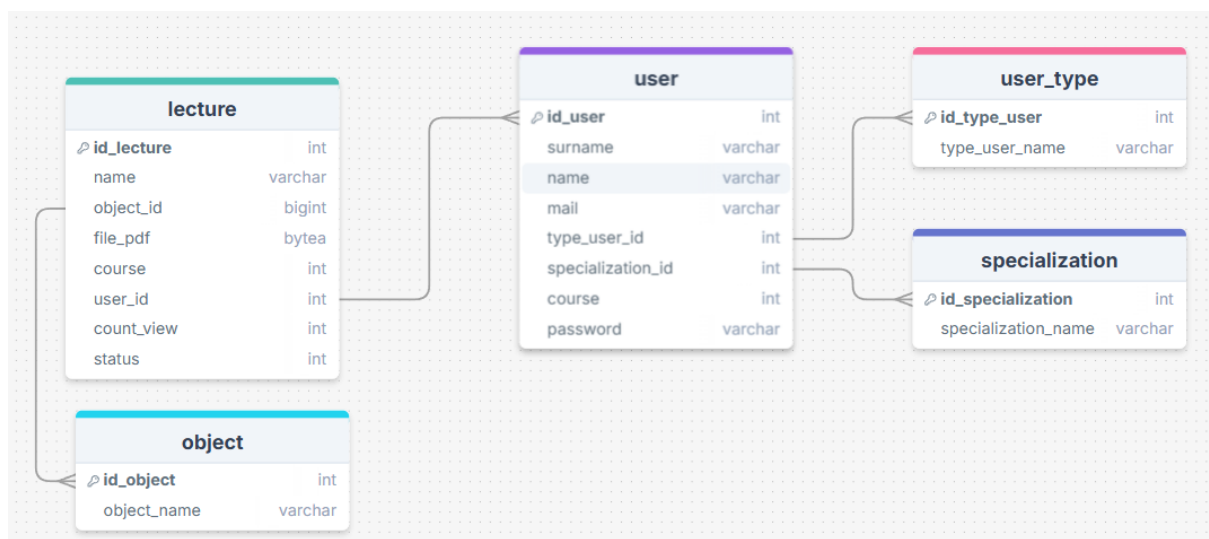


Рисунок 6 - Схема данных

Создание базы данных:

- В дереве объектов открыт раздел «Databases».
- Вызвано контекстное меню и выбран пункт "Create > Database...".
- В форме создания базы указано имя «diplom_db».
- В поле «Owner» выбран владелец базы данных «postgres».

- Нажата кнопка «Save» — база данных создана и отображается в списке.

Создание таблиц:

- Открыт раздел «Schemas > public > Tables» в структуре созданной базы данных.

- Для каждой таблицы вызвано контекстное меню и выбран пункт «Create > Table».

- Введены названия таблиц и добавлены поля:

- user_type:

- id_type_user — integer, первичный ключ

- type_user_name — varchar

- specialization:

- id_specialization — integer, первичный ключ

- specialization_name — varchar

- user:

- id_user — integer, первичный ключ

- surname, name, mail — varchar

- type_user_id — внешний ключ на user_type.id_type_user

- specialization_id — внешний ключ на specialization.id_specialization

- course — integer

- password — varchar

- object:

- id_object — integer, первичный ключ

- object_name — varchar

- lecture:

- id_lecture — integer, первичный ключ

- name — varchar

- object_id — внешний ключ на object.id_object
- file_pdf — bytea (для хранения PDF-файлов)
- course — integer
- user_id — внешний ключ на user.id_user
- count_view — integer (счётчик просмотров)

Настройка связей между таблицами:

- В каждой таблице открыта вкладка «Constraints».
- Созданы внешние ключи:
 - user.type_user_id → user_type.id_type_user
 - user.specialization_id → specialization.id_specialization
 - lecture.object_id → object.id_object
 - lecture.user_id → user.id_user

SQL запрос представлен в приложении А.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		27

3.2 Разработка пользовательского интерфейса

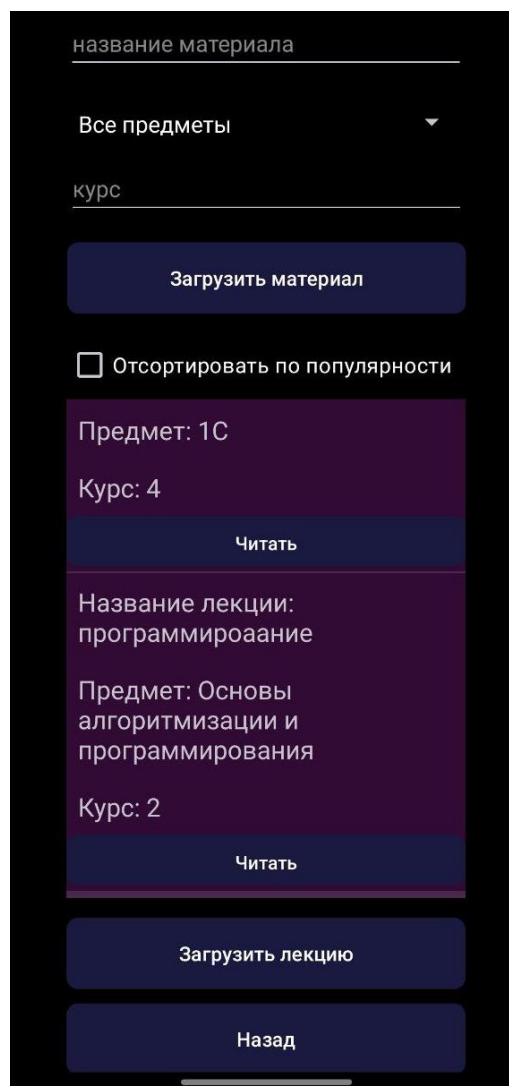


Рисунок 7 - Главное окно

На изображении, представленном на Рисунке 7, демонстрируется главное окно приложения, предназначенного для просмотра и перехода к загрузке учебных материалов, в частности лекций. Интерфейс включает в себя поле для ввода названия материала, выпадающий список с выбором предмета (по умолчанию — «Все предметы»), поле для указания курса, а также кнопку «Загрузить материал», которая используется для фильтрации лекций по введенным параметрам. Ниже расположен чекбокс «Отсортировать по популярности», позволяющий пользователю отсортировать отображаемые лекции по их популярности. В основной части экрана представлены карточки лекций, содержащие название, предмет и курс, к которому они относятся.

Каждая карточка снабжена кнопкой «Читать», которая открывает соответствующую лекцию для просмотра. В нижней части расположены две дополнительные кнопки: «Загрузить лекцию» — открывает форму для добавления нового учебного материала, и «Назад» — предназначена для возврата на предыдущий экран – экран авторизации.

Рисунок 8 - Форма для загрузки лекций

Форма для загрузки лекций, представленная на Рисунке 8 состоит из ряда элементов, расположенных вертикально. В верхней части находится поле ввода для указания названия лекции, за ним следует поле для ввода курса, к которому относится лекция. Ниже размещён выпадающий список, предназначенный для выбора предмета, по которому будет загружен материал. Далее располагается кнопка «Выбрать файл», которая открывает файловый менеджер на устройстве для выбора файла лекции. Под ней находится кнопка «Загрузить», которая отправляет все введённые данные и выбранный файл на

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		29

сервер приложения. В самом низу размещена кнопка «Назад», предназначенная для возврата на главное окно приложения без выполнения загрузки.

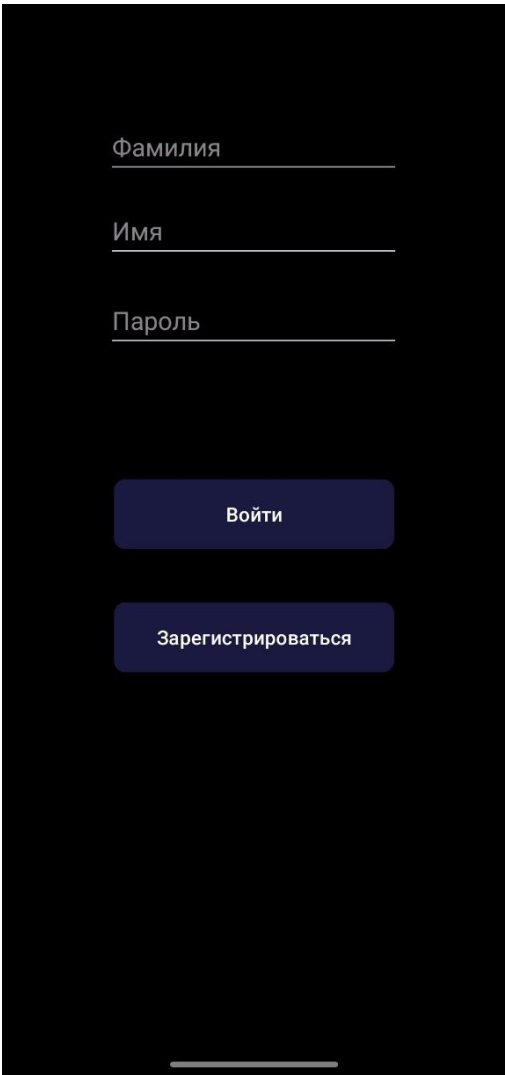


Рисунок 9 - Окно авторизации

Окно авторизации, представленное на Рисунке 9 содержит три текстовых поля, расположенных вертикально: первое предназначено для ввода фамилии пользователя, второе — для ввода имени, третье — для ввода пароля. Ниже размещены две кнопки. Кнопка «Войти» используется для выполнения входа в систему с указанными данными. Кнопка «Зарегистрироваться» предназначена для перехода к процессу регистрации нового пользователя. Все элементы размещены по центру экрана, что обеспечивает удобное взаимодействие на мобильных устройствах.

Рисунок 10 - Форма регистрации

Форма регистрации, представленная на Рисунке 10, содержит семь полей, расположенных вертикально. Первое поле предназначено для ввода фамилии пользователя, второе — для ввода имени, третье — для ввода почты. Четвёртое поле предназначено для указания курса обучения. Ниже размещены два выпадающих списка: первый используется для выбора роли пользователя, второй — для выбора направления подготовки (например, «Информационные системы и программирование»). Седьмое поле предназначено для ввода пароля. В нижней части окна расположены две кнопки: «Отмена», которая используется для выхода из формы регистрации без сохранения данных, и «Зарегистрироваться», предназначенная для отправки введённой информации и создания нового аккаунта. Все элементы интерфейса размещены по центру экрана

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		31

Визуальное оформление выполнено в тёмной цветовой гамме с использованием чёрного, тёмно-фиолетового и тёмно-синего цветов, что создаёт визуальный контраст с белым текстом и делает интерфейс комфортным для восприятия, особенно в условиях слабого освещения. Элементы интерфейса размещены вертикально, имеют чёткие границы и хорошо читаемый шрифт, что обеспечивает удобное взаимодействие с приложением на мобильных устройствах. Общий стиль минималистичен и ориентирован на функциональность, что соответствует назначению образовательного приложения.

3.2 Тестирование программного продукта

Для обеспечения высокого качества программного продукта были реализованы unit tests для ключевых элементов серверной части приложения, а именно: для класса добавления пользователя, класса добавления новой лекции, класса поиска самой популярной лекции.

Рассмотрим unit test для добавления пользователя смотреть Рисунок 11, код представлен в приложении Г.

Для успешного добавления пользователя необходимо заполнить следующие поля:

- surname (тип данных строка)
- name (тип данных строка)
- mail (тип данных строка)
- type_user_id (тип данных число)
- specialization_id (тип данных число)
- course (тип данных число)
- password (тип данных строка)

В первом тесте вместо числа 4 в параметр Course передаётся строка 'forth', что вызывает ошибку 400 с сообщением: «Course must be an integer». Это соответствует ожидаемому результату — тест считается успешно пройденным.

Во втором тесте параметр Name передаётся пустым, что приводит к ошибке 400 с сообщением: «Name is required». Сервер ожидает значение, соответствующее фамилии. Результат соответствует ожиданиям — тест пройден успешно.

В третьем тесте в параметре Password передаётся пустое значение, что вызывает ошибку 400 с сообщением: «Password cannot be empty». Сервер ожидает непустое значение пароля. Ошибка предсказуема — тест выполнен корректно.

В четвёртом тесте все параметры указаны корректно. Сервер возвращает код 200 и сообщение: «User added successful», что подтверждает успешное добавление пользователя. Тест завершён успешно.

```
((library) maksim@MacBook-Air-Maksim test % python3 add_user_t.py
/Users/maksim/Documents/studies/4_course/8_sem/diplom/test_api/start_api/library/lib/python3.9/site-packages/urllib3/_init_.py:35: NotOpenSSLWarning: urllib3 v2 only supports the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
test_add_user_invalid_course_type (__main__.TestUserAddAPI) ...
Running: test_add_user_invalid_course_type
Request Data: {'surname': 'Николаев', 'name': 'Николай', 'patronymic': 'Николаевич', 'type_user_id': 3, 'specialization_id': 1, 'course': 'fourth', 'password': '123456789'}
Response Status: 400
Response JSON: {'error': 'Course must be an integer'}
ok
test_add_user_missing_name (__main__.TestUserAddAPI) ...
Running: test_add_user_missing_name
Request Data: {'surname': 'Николаев', 'name': '', 'patronymic': 'Николаевич', 'type_user_id': 3, 'specialization_id': 1, 'course': 4, 'password': '123456789'}
Response Status: 400
Response JSON: {'error': 'Name is required'}
ok
test_add_user_missing_password (__main__.TestUserAddAPI) ...
Running: test_add_user_missing_password
Request Data: {'surname': 'Николаев', 'name': 'Николай', 'patronymic': 'Николаевич', 'type_user_id': 3, 'specialization_id': 1, 'course': 4, 'password': ''}
Response Status: 400
Response JSON: {'error': 'Password cannot be empty'}
ok
test_add_user_success (__main__.TestUserAddAPI) ...
Running: test_add_user_success
Request Data: {'surname': 'Николаев', 'name': 'Николай', 'patronymic': 'Николаевич', 'type_user_id': 3, 'specialization_id': 1, 'course': 4, 'password': '123456789'}
Response Status: 200
Response JSON: {'message': 'User added successfully'}
ok
-----
Ran 4 tests in 0.002s
OK
```

Рисунок 11 - Unit test для добавления пользователя

Перейдём к unit test для добавления лекции Рисунок 12, код представлен в приложении Б.

Для корректного выполнения запроса необходимо заполнить следующие поля

- name (тип данных строка)
- object_id (тип данных число)
- file_pdf (тип данных base64)
- course (тип данных число)
- user_id (тип данных число)

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		33

– count_view (тип данных число, по умолчанию 0)

На Рисунке 12 видно, что было выполнено 5 тестов, и все они прошли успешно. Первый тест проверяет успешное добавление лекции и получает ответ 200 с сообщением "Lecture added successfully". Второй тест проверяет обработку недопустимого base64-кода для PDF и возвращает ошибку 400 с сообщением "Invalid PDF data". Третий тест проверяет, как система реагирует на слишком большой PDF-файл, и получает ошибку 413 с сообщением "PDF file too large". Четвёртый тест проверяет отсутствие файла и получает ошибку 400 с сообщением "PDF file is required". Пятый тест проверяет, что произойдёт при отсутствии имени лекции, и также получает ошибку 400 с сообщением "Name is required". Все тесты завершились успешно, о чём говорит итоговое сообщение "Ran 5 tests in 0.035s" и статус "OK".

```
(library) maksim@MacBook-Air-Maksim test % python3 add_lecture_t.py
/Users/maksim/Documents/studies/4_course/8_sem/diplom/test_api/start_api/library/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning
he 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(
test_add_lecture_success (__main__.TestLectureAddAPI) ...
Running: test_add_lecture_success
Sent Data: {'name': 'Самостоятельная работа №1', 'object_id': 1, 'file_pdf': 'JVBERi0xLjQgZXhhbXBsZQ==', 'course': 4, 'user_id': 1, 'count_view': 0}
Response: 200 {'message': 'Lecture added successfully'}
ok
test_invalid_base64_pdf (__main__.TestLectureAddAPI) ...
Running: test_invalid_base64_pdf
Sent Data: {'name': 'Самостоятельная работа №1', 'object_id': 1, 'file_pdf': 'not_base64_data', 'course': 4, 'user_id': 1, 'count_view': 0}
Response: 400 {'error': 'Invalid PDF data'}
ok
test_large_pdf (__main__.TestLectureAddAPI) ...
Running: test_large_pdf
Sent Data: [TRUNCATED LARGE PDF]
Response: 413 {'error': 'PDF file too large'}
ok
test_missing_file (__main__.TestLectureAddAPI) ...
Running: test_missing_file
Sent Data: {'name': 'Самостоятельная работа №1', 'object_id': 1, 'course': 4, 'user_id': 1, 'count_view': 0}
Response: 400 {'error': 'PDF file is required'}
ok
test_missing_name (__main__.TestLectureAddAPI) ...
Running: test_missing_name
Sent Data: {'name': '', 'object_id': 1, 'file_pdf': 'JVBERi0xLjQgZXhhbXBsZQ==', 'course': 4, 'user_id': 1, 'count_view': 0}
Response: 400 {'error': 'Name is required'}
ok
-----
Ran 5 tests in 0.035s
OK
```

Рисунок 12 - unit test для добавления лекции

Рассмотрим заключительный unit test для поиска самой популярной лекции, код представлен в приложении В.

На рисунке 13 показан результат выполнения двух тестов, связанных с поиском самой популярной лекции. Первый тест test_find_most_server_error проверяет поведение API при внутренней ошибке сервера. Ответ имеет статус 500 и содержит сообщение об ошибке: {'error': 'Internal Server Error'}. Второй тест test_find_most_success проверяет успешный случай, когда запрос находит

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		34

самую популярную лекцию. Ответ возвращает статус 200 и JSON-объект с данными лекции: {'id': 42, 'title': 'Популярная лекция', 'views': 1234}. Оба теста завершились успешно, что подтверждается финальной строкой: Ran 2 tests in 0.001s и статусом ОК.

```
[(library) maksim@MacBook-Air-Maksim test % python3 find_most_t.py
/Users/maksim/Documents/studies/4_course/8_sem/diplom/test_api/start_api/
he 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.co
warnings.warn(
test_find_most_server_error (__main__.TestFindMostAPI) ...
Running: test_find_most_server_error
Response Status: 500
Response JSON: {'error': 'Internal Server Error'}
ok
test_find_most_success (__main__.TestFindMostAPI) ...
Running: test_find_most_success
Response Status: 200
Response JSON: {'id': 42, 'title': 'Популярная лекция', 'views': 1234}
ok

-----
Ran 2 tests in 0.001s

OK
```

Рисунок 13 - Unit test поиск самой популярной лекции

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		35

4 Инструкции по работе с АИС

4.1 Требования к техническому обеспечению

Аппаратные требования

Процессор:

- Минимально: 4-ядерный процессор с тактовой частотой ≥ 1.5 ГГц (например, Qualcomm Snapdragon 630, Apple A10 Fusion).
- Рекомендуется: 8-ядерный процессор с частотой ≥ 2.0 ГГц (например, Snapdragon 855, Apple A14 Bionic) для обработки видео и сложных операций с данными.

Оперативная память (RAM):

- Минимально: 2 ГБ для Android
- Рекомендуется: 4 ГБ и выше для многозадачности и работы с тяжелым контентом.

Внутренняя память:

- Минимально: 16 ГБ для установки приложения и базового кэша.
- Рекомендуется: 32 ГБ и выше.
- Поддержка microSD-карт: Для Android-устройств — расширение памяти до 256 ГБ (формат exFAT).

Экран:

- Разрешение: ≥ 720 p (HD) для комфортного просмотра текста.
- Диагональ: ≥ 4.7 дюймов.
- Поддержка мультитач: Для жестов.

Датчики и периферия:

- Акселерометр/гироскоп: Для автоматического поворота экрана.

Программные требования

Операционная система:

- Android: Версия 8.0 (Oreo) и выше с поддержкой API Level 26+.

Форматы файлов:

- Поддерживаемые расширения: PDF.

Сетевые требования

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№ докум.	Подпись	Дата		36

Тип соединения:

- Обязательно: Wi-Fi 802.11n/ac или мобильный интернет (4G/LTE).
- Минимальная скорость: 512 Кбит/с для загрузки документов.

Рекомендуемые устройства

Android:

- Бюджетные: Xiaomi Redmi Note 10 (4 ГБ RAM, Snapdragon 678).
- Флагманы: Samsung Galaxy S23 (8 ГБ RAM, Snapdragon 8 Gen 2).

4.2 Инструкция системного администратора, программиста

ТО-1: Ежедневное обслуживание

Порядок действий:

- Проверить доступность сервера: `ping <77.222.47.209>` или `ssh root@77.222.47.209`.
- Проверить загрузку процессора, памяти и диска: `top`, `htop`, `free -m`, `df -h`.
- Убедиться в работе PostgreSQL: `systemctl status postgresql`.
- Проверить работу API: `curl http://localhost:8000/health`.
- Просмотреть логи ошибок: `journalctl -xe`, `tail -n 100 /var/log/postgresql/postgresql.log`.
- Проверить, нет ли подозрительной активности или несанкционированных попыток входа: `last`, `auth.log`.

ТО-2: Еженедельное обслуживание

Порядок действий:

- Очистить временные и ненужные файлы: `sudo apt autoremove`, `sudo apt clean`.
- Проверить актуальность системы и ПО: `sudo apt update`, `apt list --upgradable`.
- Обновить, если требуется: `sudo apt upgrade -y`.
- Проверить активные подключения к базе(`SELECT * FROM pg_stat_activity`)

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлекеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		37

– Проверить статус cron-задач, связанных с резервным копированием: `crontab -l`.

– Проверить корректность хранения логов и их объём

ТО-3: Ежемесячное обслуживание

Порядок действий:

– Создать резервную копию базы данных(`pg_dump -U postgres -F c -b -v -f /backup/lecture_app_db_$(date +%F).backup lecture_app_db`)

– Проверить целостность и доступность бэкапов.

– Проверить файлы конфигурации:

– `pg_hba.conf` (доступ к БД)

– `postgresql.conf` (настройки производительности)

– `sshd_config` (SSH-доступ)

– Проверить настройки файервола: `sudo ufw status`.

– Анализ логов безопасности: `cat /var/log/auth.log | grep "Failed password"`.

ТО-4: Ежеквартальное обслуживание

Порядок действий:

– Оценить эффективность индексов в БД(`SELECT * FROM pg_stat_user_indexes WHERE idx_scan = 0`)

– Проверить наличие дублирующихся данных, "битых" записей.

– Очистить архивы, неактуальные или устаревшие логи.

– Провести стресс-тест приложения (с использованием Apache Benchmark или Locust).

– Обновить пароли доступа для критичных сервисов.

– Проверить работу мониторинговых систем (Prometheus).

ТО-5: Годовое техническое обслуживание

Порядок действий:

– Полное резервное копирование всей системы (БД, файлов, конфигураций).

– Обновление ОС: `do-release-upgrade` (если доступно и безопасно).

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлехеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		38

- Аудит прав пользователей, включая PostgreSQL и SSH.
- Проверка журналов аудита.
- Тестирование плана аварийного восстановления.
- Переработка или оптимизация схемы базы данных при необходимости.

- Анализ и внедрение новых технологий (если применимо)

4.3 Руководство пользователя

Общая информация

Мобильное приложение "Обучающий контент" предназначено для студентов, преподавателей и администраторов образовательных учреждений. Оно позволяет просматривать, загружать, структурировать лекционные материалы. Основная цель — предоставить удобный и быстрый доступ к образовательному контенту в цифровом формате. Схема работы приложения представлена в приложении Д.

Установка приложения

- Скачать apk файл
- Открыть
- Нажать кнопку установить

Начало работы

Регистрация (смотреть Рисунок 14)

- На стартовом экране нажать "Регистрация".
- Указать
 - Фамилию, имя, почту.
 - Пароль.
 - Тип пользователя (Студент / Преподаватель).
 - Курс и специальность (для студентов).
 - Пароль
- Нажать "Зарегистрироваться".

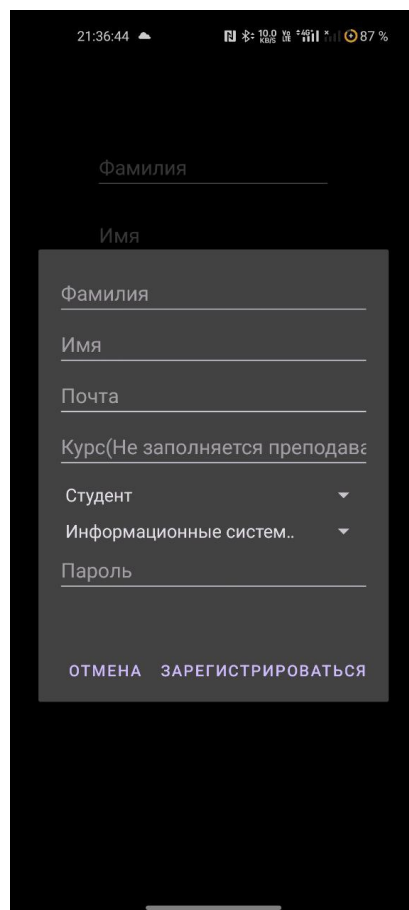


Рисунок 14 - Пример регистрации

Авторизация (смотреть Рисунок 15)

- На главном окне заполнить поля
 - Фамилия
 - Имя
 - Почта
 - Пароль
- Нажать кнопку войти

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		40

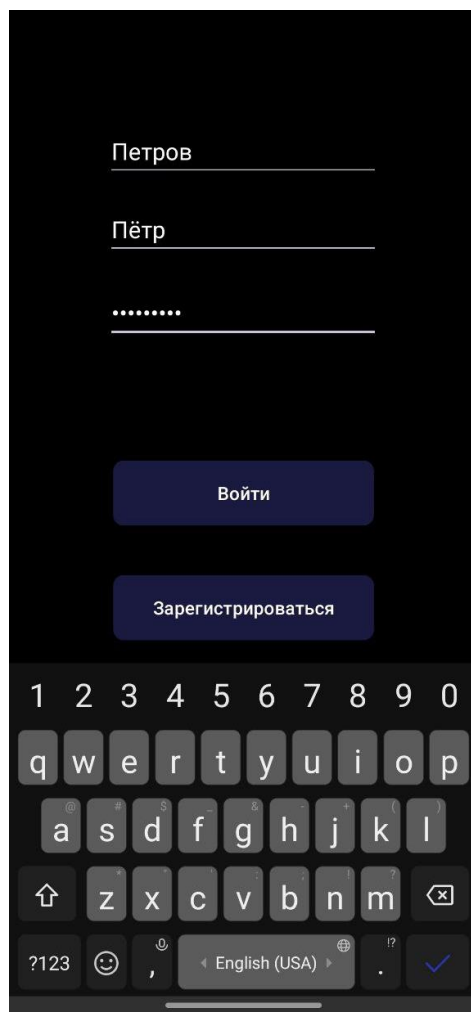


Рисунок 15 - Пример входа

Просмотр контента (Рисунок 16)

- После входа откроется главная страница с доступными лекциями.
- Можно использовать фильтр:
 - По курсу.
 - По предмету.
 - По популярности
- Также доступна строка поиска.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		41

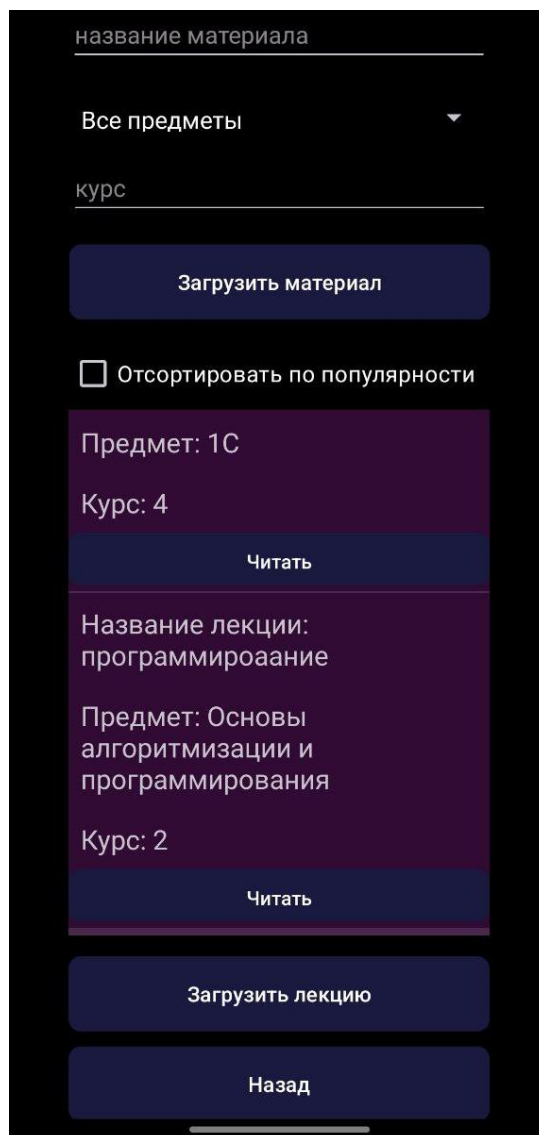


Рисунок 16 - Пример работы с главным окном

Открытие лекции (Рисунок 16 и Рисунок 17)

- Нажать на интересующую лекцию.
- Откроется PDF-файл с возможностью масштабирования и прокрутки.
- Просмотры автоматически учитываются в статистике.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		42

Практическая работа № 2

Тема Разработка технического задания и спецификации программы продукта

Цели:

1. Разработать документацию на разработку программного обеспечения;
2. Разработать систему тестирования согласно требованиям к программным продуктам.

Теоретические сведения

Техническое задание – это основной исходный документ для разработки программного обеспечения, в котором формулируются основные требования, список принципиальных требований к продукту, определены сроки и этапы разработки и регламентируется процесс приемки-сдаточных испытаний.

Этот документ содержит основные требования заказчика, исходные данные для разработки, указываются назначение продукта, области применения, стадии разработки различной документации, её состав, сроки исполнения и т. д., а также особые требования, обусловленные спецификой проекта либо условиями его эксплуатации.

Основной задачей технического задания, как документа, определяющего характеристики разрабатываемого программного обеспечения, функциональную составляющую программного обеспечения являются: исходные данные и требуемые результаты, которые определяют функции разрабатываемого программного обеспечения; среда (программная и аппаратная), в которой разрабатываемое программное обеспечение будет функционировать, может быть задано; может выбираться для обеспечения требуемых параметров; возможное взаимодействие с другим программным обеспечением.

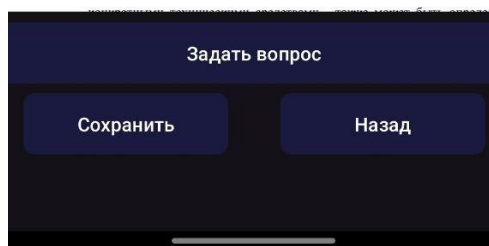


Рисунок 17 - Пример просмотра лекции

Загрузка материалов (Рисунок 18)

Нажать кнопку Загрузить лекцию

- В открывшемся окне заполнить поля
 - Название
 - Курс
 - Предмет
- Нажать кнопку выбрать файл и указать PDF файл
- Нажать загрузить

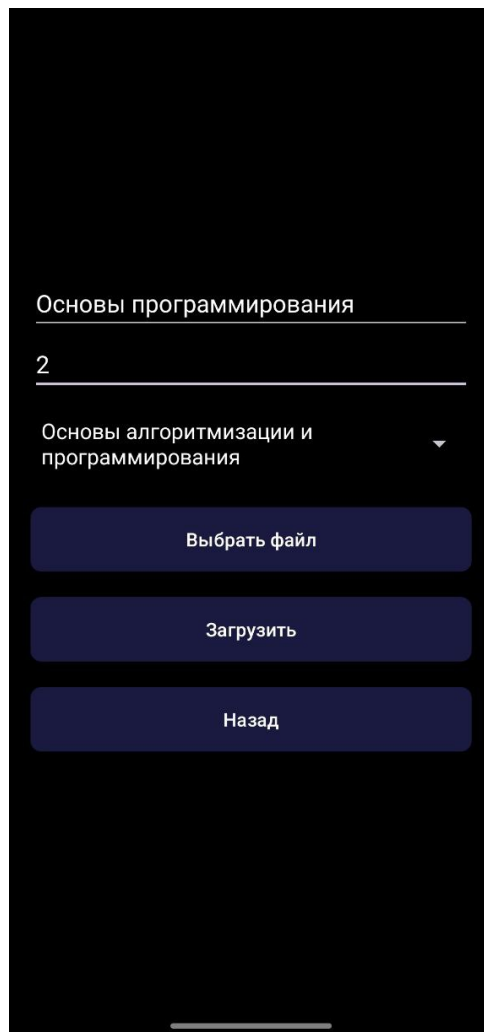


Рисунок 18 - Пример загрузки лекции

Для работы приложения необходим доступ в интернет, при возникновении ошибок связывайтесь с разработчиком.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		44

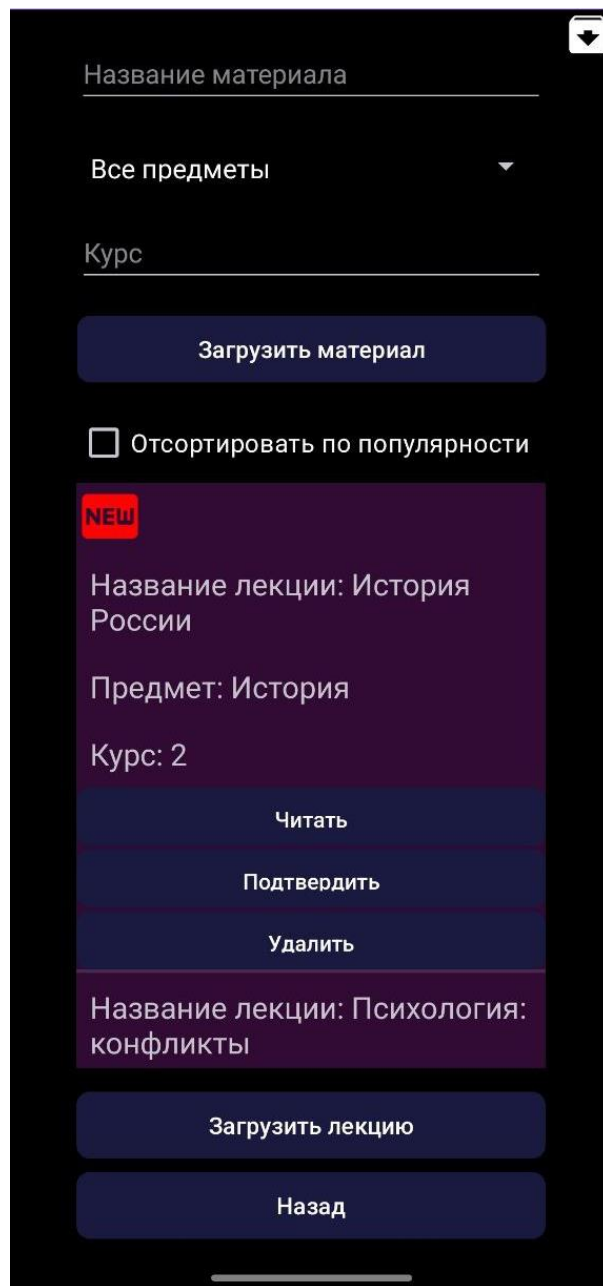


Рисунок 19 - Интерфейс администратора

Для одобрения лекции администратором необходимо просмотреть лекцию нажав кнопку «Читать», затем основываясь на увиденном материале принять решение: подтвердить или удалить и нажать на соответствующую кнопку.

Для архивирования устаревших лекций необходимо нажать на кнопку архива в правом верхнем углу.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		45

5 Охрана труда

Общие требования охраны труда при работе с мобильными устройствами

К работе с мобильными устройствами (смартфоны, планшеты) допускаются люди, прошедшие вводный и первичный инструктаж по охране труда, а также ознакомленные с правилами эксплуатации конкретных устройств.

Опасные и вредные производственные факторы:

- Физические:
 - Повышенная нагрузка на зрительный анализатор из-за мелкого шрифта, бликов, мерцания экрана.
 - Статическое напряжение мышц шеи, плечевого пояса, кистей рук при длительном удержании устройства.
 - Риск перегрева аккумулятора, ведущий к возгоранию или взрыву.
 - Электромагнитное излучение (в пределах допустимых норм, но требующее контроля).
- Эргономические:
 - Неправильная поза при использовании, приводящая к развитию остеохондроза, туннельного синдрома.
 - Психологическое напряжение из-за многозадачности и постоянных уведомлений.
- Технические:
 - Короткое замыкание при использовании неисправных зарядных устройств.
 - Повреждение устройства из-за падений, ударов или попадания влаги.

Требования к работнику:

- Соблюдать режим труда и отдыха:
 - Непрерывная работа с экраном не должна превышать 40 минут; обязателен перерыв на 20–30 минут для гимнастики глаз и мышц.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		46

- Общая продолжительность работы с мобильным устройством — не более 6 часов в день (с учетом перерывов).
- Использовать эргономичные аксессуары:
 - Подставки для фиксации устройства на уровне глаз.
 - Внешние клавиатуры и мыши для снижения нагрузки на кисти.
 - Антибликовые пленки и фильтры синего света для защиты зрения.
- Следить за исправностью оборудования:
 - Не использовать устройства с поврежденным корпусом, вздутыми аккумуляторами или перегревающимися компонентами.
 - Избегать зарядки на мягких поверхностях (диваны, кровати), которая блокирует приток воздуха.

Организация рабочего места:

- Освещение:
 - Экран должен быть защищен от прямых солнечных лучей и бликов.
 - Рекомендуются использовать рассеянный свет с интенсивностью 300–500 люкс.
- Эргономика:
 - Устройство размещается на расстоянии 40–50 см от глаз.
 - Угол наклона экрана — 15–30 градусов для минимизации напряжения шеи.
- Безопасность:
 - Избегать работы вблизи источников воды, в условиях высокой влажности или запыленности.
 - Не оставлять устройство без присмотра в общественных местах.

Требования перед началом работы:

- Проверка оборудования:
 - Убедиться в отсутствии трещин на экране, повреждений корпуса, коррозии на контактах.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлехеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		47

- Проверить работу кнопок и сенсорного ввода.
- Подготовка среды:
 - Очистить экран от пыли и следов пальцев специальными салфетками.
 - Настроить яркость и контрастность экрана в соответствии с освещением.
- Зарядка:
 - Использовать только оригинальные или сертифицированные зарядные устройства.
 - Не подключать устройство к сети при наличии признаков неисправности (искрение, запах гари).

Запрещено:

- Работать с устройством, если аккумулятор нагревается выше 45°C.
- Использовать поврежденные кабели или адаптеры.

Требования во время работы:

- Гигиена труда:
 - Каждые 20–30 минут выполнять упражнения для глаз: фокусировка на удаленных объектах, круговые движения глазами яблоками.
 - Каждые 40 минут делать перерыв на разминку: повороты шеи, вращения плечами, растяжку кистей.
- Электробезопасность:
 - Не допускать перекручивания или передавливания кабелей.
 - Отключать устройство от сети при первых признаках перегрева.
- Правила эксплуатации:
 - Не использовать устройство во время движения (ходьба, управление транспортом).
 - Избегать длительных разговоров без гарнитуры для снижения воздействия электромагнитного излучения.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№ докум.	Подпись	Дата		48

Запрещено:

- Заряжать устройство под подушкой или в закрытых пространствах.
- Устанавливать нелицензионное ПО, повышающее риск перегрузки системы.

Действия в аварийных ситуациях:

- Перегрев или возгорание аккумулятора:
 - Немедленно отключить устройство от сети.
 - Накрыть источник возгорания плотной тканью или использовать огнетушитель класса ABC.
 - Эвакуироваться из помещения и сообщить руководству.
- Попадание влаги:
 - Выключить устройство, извлечь аккумулятор (если возможно).
 - Протереть корпус сухой тканью и поместить в абсорбирующее вещество (силикагель).
- Поражение электрическим током:
 - Отключить питание через рубильник.
 - Оказать первую помощь: искусственное дыхание, непрямой массаж сердца (при необходимости).

Требования по окончании работы:

- Завершение сеанса:
 - Закрыть все приложения, сохранить данные.
 - Отключить Bluetooth, Wi-Fi и мобильный интернет для экономии заряда.
- Уход за устройством:
 - Очистить экран и корпус от загрязнений.
 - Убрать устройство в защитный чехол или сумку.
- Личная безопасность:

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		49

- Выполнить комплекс упражнений для расслабления мышц спины, шеи и кистей.
- Избегать использования устройства в постели для профилактики бессонницы.

Дополнительные рекомендации:

- Использовать приложения для контроля времени работы (например, цифровой детокс).
- Регулярно проходить медосмотры для выявления ранних признаков профессиональных заболеваний (синдром сухого глаза, туннельный синдром).
- При работе на улице применять антибликовые экраны и power-bank для исключения форсированной зарядки.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		50

Заключение

В ходе проделанной работы была реализована информационная система в виде мобильного приложения под названием "Обучающий контент", основной задачей которого является удобное и эффективное предоставление лекционного материала в цифровом формате. Разработка охватывает все ключевые аспекты современного образовательного процесса — от хранения и структурирования информации до её доступного отображения и возможности интерактивного взаимодействия с контентом.

Приложение включает ряд важнейших функций, таких как регистрация и авторизация пользователей, загрузка лекций, их просмотр в удобном и читаемом виде, а также фильтрация, поиск и оценка материалов. Кроме того, предусмотрена возможность архивирования устаревших лекций, что позволяет поддерживать актуальность контента и упорядочивать информационное пространство приложения.

Для реализации серверной части проекта была выбрана надёжная и широко применяемая система управления базами данных — PostgreSQL, установленная на удалённый сервер под управлением операционной системы Ubuntu Server. После настройки сервера и обеспечения удалённого доступа к СУБД, база данных была создана и структурирована с использованием графического интерфейса PgAdmin, что значительно упростило процесс её проектирования и настройки.

В результате созданная система обеспечивает централизованное и надёжное хранилище образовательных материалов, улучшает взаимодействие между студентами и преподавателями, а также способствует оптимизации учебного процесса за счёт использования современных технологий. Работа охватила все этапы жизненного цикла информационной системы — от анализа предметной области и проектирования базы данных до практической реализации приложения и настройки серверной инфраструктуры. Всё это говорит о высокой степени завершённости и прикладной значимости разработанного решения.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		51

Список информационных источников

- 1) Android Studio [Электронный ресурс]. – Режим доступа: <https://developer.android.com/studio/>. – Дата обращения: 05.04.2025.
- 2) Алгоритм работы информационной системы [Электронный ресурс]. – Режим доступа: <https://studentopedia.ru/informatika/algorithm-raboti-informacionnoj-sistemi--razrabotka-interfejsa-polzovatelya---razrabotka.html/>. – Дата обращения: 10.04.2025.
- 3) База данных Firebase: что это, зачем нужна и чем хороша [Электронный ресурс]. – Режим доступа: <https://skillbox.ru/media/code/baza-dannykh-firebase-chto-eto-zachem-nuzhna-i-kak-ispolzovat/>. – Дата обращения: 12.04.2025.
- 4) Бейзер Б. Тестирование черного ящика. – 2019 г. – 95 с.
- 5) Васильев А. М. Программирование на Java для начинающих. – 2023 г. – С. 205–210.
- 6) Веремеенко А. В. SQLite и Android. Управление и разработка. – 2021 г. – 480 с.
- 7) Гончаренко С. Д. Android. Руководство разработчика. – 2022 г. – С. 150–903.
- 8) Тидвелл Дж. Разработка дизайна пользовательского интерфейса. – 2021 г. – 326 с.
- 9) Шарп Дж. Java для Android. Полное руководство. – 2020 г. – С. 10–107.
- 10) Инструкция по охране труда при работе с персональным компьютером [Электронный ресурс]. – Режим доступа: <https://ohrana-tryda.com/node/10/>. – Дата обращения: 15.04.2025.
- 11) Казарин О. В. Основы информационной безопасности: надежность и безопасность мобильных приложений. – 2020 г. – С. 91–100.
- 12) Леоненков А. В. Самоучитель UML. – 2019 г. – С. 100–107.

13) Министерство труда занятости и социальной защиты Республики Татарстан [Электронный ресурс]. – Режим доступа: <https://mtsz.tatarstan.ru/>. – Дата обращения: 20.04.2025.

14) Надежность мобильных информационных систем: защита данных и обеспечение бесперебойной работы [Электронный ресурс]. – Режим доступа: <https://nauchniestati.ru/spravka/nadezhnost-informacionnyh-sistem/>. – Дата обращения: 25.04.2025.

15) Пользовательский интерфейс мобильных приложений [Электронный ресурс]. – Режим доступа: <https://practicum.yandex.ru/blog/chtotakoe-polzovatelskii-interfeys/>. – Дата обращения: 27.04.2025.

16) Тестирование методом чёрного ящика [Электронный ресурс]. – Режим доступа: <https://tquality.ru/blog/testirovanie-metodom-chernogo-yashchika/>. – Дата обращения: 30.04.2025.

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№. докум.	Подпись	Дата		53

Приложение А

```

CREATE TABLE [dbo].[favotite_lecture](
    [id_favorite_lecture] [int] NOT NULL,
    [lecture_id] [int] NOT NULL,
    [user_id] [int] IDENTITY(1,1) NOT
NULL,
    CONSTRAINT [PK_favotite_lecture]
PRIMARY KEY CLUSTERED
(
    [id_favorite_lecture] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON,
OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
/***** Object: Table [dbo].[lecture] Script
Date: 23.01.2025 9:15:27 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[lecture](
    [id_lecture] [int] IDENTITY(1,1) NOT
NULL,
    [name] [nvarchar](100) NOT NULL,
    [object_id] [int] NOT NULL,
    [file_pdf] [binary](8000) NOT NULL,
    [course] [int] NOT NULL,
    [user_id] [int] NOT NULL,
    [count_view] [int] NOT NULL,
    CONSTRAINT [PK_lecture] PRIMARY KEY
CLUSTERED
(
    [id_lecture] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON,
OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
/***** Object: Table [dbo].[specialization]
Script Date: 23.01.2025 9:15:27 *****/
SET ANSI_NULLS ON
GO
CREATE TABLE [dbo].[object](
    [id_object] [int] IDENTITY(1,1) NOT
NULL,
    [object_name] [nvarchar](100) NOT
NULL,
    CONSTRAINT [PK_object] PRIMARY KEY
CLUSTERED
(
    [id_object] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON,
OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
/***** Object: Table [dbo].[specialization]
Script Date: 23.01.2025 9:15:27 *****/
SET ANSI_NULLS ON
GO

```

```

SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[specialization](
    [id_specialization] [int] IDENTITY(1,1)
    NOT NULL,
    [specialization_name] [nvarchar](150)
    NOT NULL,
    CONSTRAINT [PK_specialization] PRIMARY
    KEY CLUSTERED
(
    [id_specialization] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON,
OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
/***** Object: Table [dbo].[user] Script Date:
23.01.2025 9:15:27 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[user](
    [id_user] [int] IDENTITY(1,1) NOT
    NULL,
    [surname] [nvarchar](50) NOT NULL,
    [name] [nvarchar](50) NOT NULL,
    [patronymic] [nvarchar](50) NULL,
    [type_user_id] [int] NOT NULL,
    [specialization_id] [int] NOT NULL,
    [course] [int] NOT NULL,
    CONSTRAINT [PK_user] PRIMARY KEY
    CLUSTERED
(

```

```

[id_user] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON,
OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
/***** Object: Table [dbo].[user_type] Script
Date: 23.01.2025 9:15:27 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[user_type](
    [id_type_user] [int] IDENTITY(1,1)
    NOT NULL,
    [type_user_name] [nvarchar](50) NOT
    NULL,
    CONSTRAINT [PK_user_type] PRIMARY
    KEY CLUSTERED
(
    [id_type_user] ASC
)WITH (PAD_INDEX = OFF,
STATISTICS_NORECOMPUTE = OFF,
IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON,
OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF)
ON [PRIMARY]
) ON [PRIMARY]
GO
ALTER TABLE [dbo].[lecture] ADD
CONSTRAINT [DF_lecture_count_view]
DEFAULT ((0)) FOR [count_view]
GO

```

```

ALTER TABLE [dbo].[favotite_lecture] WITH
CHECK      ADD      CONSTRAINT
[FK_favotite_lecture_lecture]      FOREIGN
KEY([lecture_id])
REFERENCES [dbo].[lecture] ([id_lecture])
GO

ALTER TABLE [dbo].[favotite_lecture] CHECK
CONSTRAINT [FK_favotite_lecture_lecture]
GO

ALTER TABLE [dbo].[favotite_lecture] WITH
CHECK      ADD      CONSTRAINT
[FK_favotite_lecture_user]      FOREIGN
KEY([user_id])
REFERENCES [dbo].[user] ([id_user])
GO

ALTER TABLE [dbo].[favotite_lecture] CHECK
CONSTRAINT [FK_favotite_lecture_user]
GO

ALTER TABLE [dbo].[lecture] WITH CHECK
ADD      CONSTRAINT [FK_lecture_object]
FOREIGN KEY([object_id])
REFERENCES [dbo].[object] ([id_object])
GO

ALTER TABLE [dbo].[lecture] CHECK
CONSTRAINT [FK_lecture_object]
GO

ALTER TABLE [dbo].[lecture] WITH CHECK
ADD      CONSTRAINT [FK_lecture_user]
FOREIGN KEY([user_id])
REFERENCES [dbo].[user] ([id_user])
GO

ALTER TABLE [dbo].[lecture] CHECK
CONSTRAINT [FK_lecture_user]
GO

ALTER TABLE [dbo].[user] WITH CHECK
ADD CONSTRAINT [FK_user_specialization]
FOREIGN KEY([specialization_id])

```

```

REFERENCES      [dbo].[specialization]
([id_specialization])
GO

ALTER TABLE [dbo].[user] CHECK
CONSTRAINT [FK_user_specialization]
GO

ALTER TABLE [dbo].[user] WITH CHECK
ADD      CONSTRAINT [FK_user_user_type]
FOREIGN KEY([type_user_id])
REFERENCES      [dbo].[user_type]
([id_type_user])
GO

ALTER TABLE [dbo].[user] CHECK
CONSTRAINT [FK_user_user_type]
GO

USE [master]
GO

ALTER DATABASE [MAX] SET
READ_WRITE
GO

```


Приложение Б

```

import unittest
from unittest.mock import patch
import requests
import base64

class TestLectureAddAPI(unittest.TestCase):
    def setUp(self):
        self.url = "http://77.222.47.209:3001/api/lecture_add"
        self.sample_pdf_data = b"%PDF-1.4 example"
        self.pdf_base64 = base64.b64encode(self.sample_pdf_data).decode("utf-8")

        self.valid_data = {
            "name": "Самостоятельная работа №1",
            "object_id": 1,
            "file_pdf": self.pdf_base64,
            "course": 4,
            "user_id": 1,
            "count_view": 0
        }

    @patch('requests.put')
    def test_add_lecture_success(self, mock_put):
        print("\nRunning: test_add_lecture_success")
        mock_put.return_value.status_code = 200
        mock_put.return_value.json.return_value = {"message": "Lecture added successfully"}

        response = requests.put(self.url, json=self.valid_data)
        print("Sent Data:", self.valid_data)

        print("Response:", response.status_code, response.json())

        self.assertEqual(response.status_code, 200)
        self.assertIn("Lecture added successfully", response.json()["message"])

    @patch('requests.put')
    def test_missing_name(self, mock_put):
        print("\nRunning: test_missing_name")
        data = self.valid_data.copy()
        data["name"] = ""
        mock_put.return_value.status_code = 400
        mock_put.return_value.json.return_value = {"error": "Name is required"}

        response = requests.put(self.url, json=data)
        print("Sent Data:", data)
        print("Response:", response.status_code, response.json())

        self.assertEqual(response.status_code, 400)
        self.assertIn("Name is required", response.json()["error"])

    @patch('requests.put')
    def test_invalid_base64_pdf(self, mock_put):
        print("\nRunning: test_invalid_base64_pdf")
        data = self.valid_data.copy()
        data["file_pdf"] = "not_base64_data"
        mock_put.return_value.status_code = 400
        mock_put.return_value.json.return_value = {"error": "Invalid PDF data"}

        response = requests.put(self.url, json=data)
        print("Sent Data:", data)

```

```

        print("Response:", response.status_code,
response.json())

```

```

        self.assertEqual(response.status_code, 400)
        self.assertIn("Invalid PDF data",
response.json()["error"])

```

```

@patch('requests.put')
def test_missing_file(self, mock_put):
    print("\nRunning: test_missing_file")
    data = self.valid_data.copy()
    del data["file_pdf"]
    mock_put.return_value.status_code = 400
    mock_put.return_value.json.return_value =
{"error": "PDF file is required"}

```

```

    response = requests.put(self.url, json=data)
    print("Sent Data:", data)
    print("Response:", response.status_code,
response.json())

```

```

    self.assertEqual(response.status_code, 400)
    self.assertIn("PDF file is required",
response.json()["error"])

```

```

@patch('requests.put')

```

```

def test_large_pdf(self, mock_put):

```

```

    print("\nRunning: test_large_pdf")

```

```

    data = self.valid_data.copy()

```

```

    large_pdf_data = base64.b64encode(b"A" *

```

```

10_000_000).decode("utf-8") # 10 MB

```

```

    data["file_pdf"] = large_pdf_data

```

```

    mock_put.return_value.status_code = 413

```

```

    mock_put.return_value.json.return_value =
{"error": "PDF file too large"}

```

```

    response = requests.put(self.url, json=data)

```

```

    print("Sent Data: [TRUNCATED LARGE
PDF]")

```

```

    print("Response:", response.status_code,
response.json())

```

```

    self.assertEqual(response.status_code, 413)

```

```

    self.assertIn("PDF file too large",
response.json()["error"])

```

```

if __name__ == '__main__':

```

```

    unittest.main(verbosity=2)

```

Приложение В

```

import unittest
from unittest.mock import patch
import requests

class TestFindMostAPI(unittest.TestCase):
    def setUp(self):
        self.url = "http://127.0.0.1:3000/api/find_most"

    @patch('requests.get')
    def test_find_most_server_error(self, mock_get):
        mock_get.return_value.status_code = 500
        mock_get.return_value.json.return_value = {"error": "Internal Server Error"}

        response = requests.get(self.url)
        print("Response Status:", response.status_code)
        print("Response JSON:", response.json())

        self.assertEqual(response.status_code, 500)
        self.assertIn("Internal Server Error", response.json()["error"])

    @patch('requests.get')
    def test_find_most_success(self, mock_get):
        print("\nRunning: test_find_most_success")
        mock_data = {
            "id": 42,
            "title": "Популярная лекция",
            "views": 1234
        }
        mock_get.return_value.status_code = 200
        mock_get.return_value.json.return_value = mock_data

        response = requests.get(self.url)
        print("Response Status:", response.status_code)
        print("Response JSON:", response.json())

```

Приложение Г

```

import unittest
from unittest.mock import patch
import requests

class TestUserAddAPI(unittest.TestCase):
    def setUp(self):
        self.url =
"http://77.222.47.209:3001/api/user_add"
        self.valid_data = {
            "surname": "Николаев",
            "name": "Николай",
            "patronymic": "Николаевич",
            "type_user_id": 3,
            "specialization_id": 1,
            "course": 4,
            "password": "123456789"
        }

    @patch('requests.put')
    def test_add_user_success(self, mock_put):
        print("\nRunning: test_add_user_success")
        mock_put.return_value.status_code = 200
        mock_put.return_value.json.return_value =
{"message": "User added successfully"}

        response = requests.put(self.url,
json=self.valid_data)
        print("Request Data:", self.valid_data)
        print("Response Status:",
response.status_code)
        print("Response JSON:", response.json())

        self.assertEqual(response.status_code, 200)
        self.assertEqual(response.json(),
{"message": "User added successfully"})

    @patch('requests.put')
    def test_add_user_missing_name(self,
mock_put):
        print("\nRunning:
test_add_user_missing_name")
        data = self.valid_data.copy()
        data["name"] = ""
        mock_put.return_value.status_code = 400
        mock_put.return_value.json.return_value =
{"error": "Name is required"}

        response = requests.put(self.url, json=data)
        print("Request Data:", data)
        print("Response Status:",
response.status_code)
        print("Response JSON:", response.json())

        self.assertEqual(response.status_code, 400)
        self.assertEqual(response.json()["error"],
"Name is required")

    @patch('requests.put')
    def test_add_user_invalid_course_type(self,
mock_put):
        print("\nRunning:
test_add_user_invalid_course_type")
        data = self.valid_data.copy()
        data["course"] = "fourth" # string instead
of int
        mock_put.return_value.status_code = 400
        mock_put.return_value.json.return_value =
{"error": "Course must be an integer"}

        response = requests.put(self.url, json=data)
        print("Request Data:", data)
        print("Response Status:",
response.status_code)
        print("Response JSON:", response.json())

```

		Случаев М.К.			ДП 09.02.07.21И1.16.000 ПЗ	Лист
		Мавлеева Л. И.				
Изм.	Лист	№ докум.	Подпись	Дата		60

```

self.assertEqual(response.status_code, 400)
self.assertIn("Course must be an integer",
response.json()["error"])

```

```

@patch('requests.put')
def test_add_user_missing_password(self,
mock_put):
    print("\nRunning:
test_add_user_missing_password")
    data = self.valid_data.copy()
    data["password"] = ""
    mock_put.return_value.status_code = 400
    mock_put.return_value.json.return_value =
{"error": "Password cannot be empty"}

```

```

response = requests.put(self.url, json=data)
print("Request Data:", data)
print("Response Status:",
response.status_code)
print("Response JSON:", response.json())

```

```

self.assertEqual(response.status_code, 400)
self.assertIn("Password cannot be empty",
response.json()["error"])

```

```

if __name__ == '__main__':
    unittest.main(verbosity=2)

```

Приложение Д

